SOLID Principles in C#

SOLID Overview

SOLID is an acronym for five design principles intended to make software designs more understandable, flexible, and maintainable.

- **S** - Single Responsibility Principle
- **O** - Open/Closed Principle
- **L** - Liskov Substitution Principle
- **I** - Interface Segregation Principle
- **D** - Dependency Inversion Principle

Each principle addresses a specific design concern, helping developers avoid common pitfalls like tightly coupled code, fragile hierarchies, and bloated classes.

PRINCIPLE 1

Single Responsibility Principle (SRP)

A class should have only *one reason to change*. Each class should be responsible for a single, well-defined piece of functionality.

❌ Before — Violating SRP

One class doing everything: data storage, reporting, and email.

```
public class Employee
{
  public string Name { get; set; }
  public decimal Salary { get; set; }

  // DB, Report, Email — 3 reasons to change!
  public void SaveToDatabase() { ... }
  public void GenerateReport() { ... }
  public void SendEmail() { ... }
}
```

✅ After — Following SRP

Each responsibility lives in its own class.

```
public class Employee
{
  public string Name { get; set; }
  public decimal Salary { get; set; }
}

public class EmployeeRepository
{
  public void SaveToDatabase(Employee e) { ... }
}

public class EmployeeReportGenerator
{
  public void GenerateReport(Employee e) { ... }
}

public class EmployeeEmailService
{
  public void SendEmail(Employee e) { ... }
}
```

PRINCIPLE 2

Open/Closed Principle (OCP)

Software entities should be *open for extension* but *closed for modification*. Add new behavior without changing existing code.

❌ Before — Violating OCP

Adding a new shape requires modifying this class every time.

```
public class AreaCalculator
{
  public double CalculateArea(object shape)
  {
    if (shape is Rectangle)
    {
      var r = (Rectangle)shape;
      return r.Width * r.Height;
    }
    else if (shape is Circle)
    {
      var c = (Circle)shape;
      return Math.PI * c.Radius * c.Radius;
    }
    // Modify for every new shape!
    return 0;
  }
}
```

✅ After — Following OCP

New shapes implement `IShape` — no changes needed to `AreaCalculator`.

```
public interface IShape
{
 double CalculateArea();
}

public class Rectangle : IShape
{
 public double Width { get; set; }
 public double Height { get; set; }
 public double CalculateArea() => Width * Height;
}

public class Circle : IShape
{
 public double Radius { get; set; }
 public double CalculateArea() => Math.PI * Radius * Radius;
}

// No modification needed for new shapes!
public class AreaCalculator
{
 public double CalculateArea(IShape shape) => shape.CalculateArea();
}

// Usage
var calculator = new AreaCalculator();

var rectangle = new Rectangle { Width = 5.0, Height = 4.0 };
var circle = new Circle { Radius = 3.0 };

Console.WriteLine(calculator.CalculateArea(rectangle));
Console.WriteLine(calculator.CalculateArea(circle));
```

PRINCIPLE 3

Liskov Substitution Principle (LSP)

Objects of a superclass shall be replaceable with objects of subclasses *without affecting correctness*. Derived classes must be substitutable for their base classes.

❌ Before — Violating LSP

An `Ostrich` is a `Bird`, but it can't fly - breaks substitution.

```
public class Bird
{
  public virtual void Fly() { ... }
}

public class Ostrich : Bird
{
  // Violation: Ostrich can't fly!
  public override void Fly()
  {
    throw new InvalidOperationException("Ostrich cannot fly!");
  }
}
```

✅ After — Following LSP

Separate interfaces ensure only flyable birds are passed to fly methods.

```
public interface IFlyable
{
 void Fly();
}

public interface IRunnable
{
 void Run();
}

public class Sparrow : IFlyable
{
 public void Fly() { ... }
}

public class Ostrich : IRunnable
{
 public void Run() { ... }
}

// Safe substitution — only flyable birds
public void MakeBirdFly(IFlyable bird)
{
 bird.Fly(); // Safe!
}
```

PRINCIPLE 4

Interface Segregation Principle (ISP)

Clients should not be forced to depend on interfaces they *do not use*. Split large interfaces into smaller, focused ones.

❌ Before — Violating ISP

A `Robot` is forced to implement `Eat()` and `Sleep()` it doesn't need.

```
public interface IWorker
{
  void Work();
  void Eat();
  void Sleep();
}

public class Robot : IWorker
{
  public void Work() { ... }
  // Violation: Robot doesn't need these!
  public void Eat() { /* throw */ }
  public void Sleep() { /* throw */ }
}
```

✅ After — Following ISP

Small, focused interfaces - each class implements only what it needs.

```
public interface IWorkable
{
  void Work();
}

public interface IFeedable
{
  void Eat();
}

public interface ISleepable
{
  void Sleep();
}

public class Human : IWorkable, IFeedable, ISleepable
{
  public void Work() { ... }
  public void Eat() { ... }
  public void Sleep() { ... }
}

public class Robot : IWorkable
{
  public void Work() { ... }
}
```

PRINCIPLE 5

Dependency Inversion Principle (DIP)

High-level modules should not depend on low-level modules; *both should depend on abstractions*.

Depend on abstractions, not concretions.

❌ Before — Violating DIP

`OrderService` is tightly coupled to `SqlDatabase`.

```
public class OrderService
{
  private readonly SqlDatabase _database;

  public OrderService()
  {
    _database = new SqlDatabase(); // Tight coupling!
  }

  public void SaveOrder(Order order)
  {
    _database.Save(order);
  }
}
```

✅ After — Following DIP

Depend on `IDatabase` abstraction - swap implementations freely.

```
public interface IDatabase
{
 void Save(object entity);
}

public class SqlDatabase : IDatabase
{
 public void Save(object e) { ... }
}

public class MongoDatabase : IDatabase
{
 public void Save(object e) { ... }
}

// High-level module depends on abstraction
public class OrderService
{
 private readonly IDatabase _database;

 public OrderService(IDatabase database)
 {
  _database = database; // Dependency injection
 }

 public void SaveOrder(Order order)
 {
  _database.Save(order);
 }
}

// Usage
var service = new OrderService(new SqlDatabase());
// or
var service = new OrderService(new MongoDatabase());
```

PRACTICAL EXAMPLE

E-Commerce Order Processing: Before Refactoring

Consider a real-world **E-Commerce Order Processing System**. Below is a monolithic `OrderProcessor` class that handles everything - a textbook example of multiple SOLID violations.

The Problem

This single class violates **SRP** (too many responsibilities), **OCP** (hard-coded payment logic), **ISP** (no interfaces), and **DIP** (concrete dependencies). Any change - adding a new payment method, changing validation rules, or switching the database - requires modifying this class directly.

❌ Monolithic OrderProcessor

```
public class OrderProcessor
{
  public void Process(Order order)
  {
    // Validate
    if (order.Items.Count == 0) throw ...

    // Check inventory
    if (!Inventory.InStock(order)) throw ...

    // Process payment (hard-coded!)
    if (order.PaymentType == "Credit") { ... }
    else if (order.PaymentType == "PayPal") { ... }

    // Save to DB
    new SqlDatabase().Save(order);

    // Send email
    new EmailService().Send(order);
  }
}
```

PRACTICAL EXAMPLE

Refactoring Step-by-Step

Applying each SOLID principle systematically transforms the monolithic `OrderProcessor` into a clean, maintainable architecture.

01

Apply SRP

Split into `OrderValidator`, `PaymentProcessor`, `InventoryManager`, and `NotificationService` — each with a single responsibility.

02

Apply OCP

Use the **Strategy Pattern** for different payment methods. Add new payment types without modifying existing code.

03

Apply LSP

Ensure all payment processors are substitutable. Any `IPaymentProcessor` can be swapped without breaking behavior.

04

Apply ISP

Define separate, focused interfaces: `IOrderValidator`, `IPaymentProcessor`, `IInventoryService`, `INotificationService`.

05

Apply DIP

Inject dependencies via constructor. Use a DI container to wire up concrete implementations at runtime.

AFTER REFACTORING

Clean, Maintainable Code Structure

After applying all five SOLID principles, the system is modular, testable, and easy to extend.

Interfaces

```
public interface IOrderValidator
{
  ValidationResult Validate(Order order);
}

public interface IPaymentProcessor
{
  PaymentResult Process(Order order);
}

public interface IInventoryService
{
  bool CheckAvailability(Order order);
}

public interface INotificationService
{
  void SendConfirmation(Order order);
}
```

Refactored OrderProcessor

```
public class OrderProcessor
{
  private readonly IOrderValidator _validator;
  private readonly IPaymentProcessor _payment;
  private readonly IInventoryService _inventory;
  private readonly INotificationService _notify;

  public OrderProcessor(
    IOrderValidator validator,
    IPaymentProcessor payment,
    IInventoryService inventory,
    INotificationService notify)
  {
    _validator = validator;
    _payment = payment;
    _inventory = inventory;
    _notify = notify;
  }

  public void Process(Order order)
  {
    _validator.Validate(order);
    _inventory.CheckAvailability(order);
    _payment.Process(order);
    _notify.SendConfirmation(order);
  }
}
```

Benefits of SOLID

Easier Maintenance

Changes are localized - modify one class without breaking others.

Better Testability

Small, focused classes with injected dependencies are easy to unit test.

Reduced Coupling

Abstractions and interfaces decouple modules, reducing ripple effects.

Increased Reusability

Single-purpose classes can be reused across different parts of the system.

Scalable Architecture

New features are added by extending, not modifying - the system grows gracefully.

Common Pitfalls & Best Practices

Don't Over-Engineer

Apply SOLID when complexity demands it - not prematurely.

YAGNI — You Aren't Gonna Need It

Don't build abstractions for hypothetical future requirements.

Start Simple, Refactor When Needed

Write working code first, then refactor as patterns emerge.

Unit Testing Validates SOLID

If code is hard to test, it likely violates SOLID principles.